01/11/2026 11:30 AM

Your PDF roadmap

One honest constraint first: I can't attach a binary .pdf file in this chat. What I can do — and what works better anyway — is give you the complete document in a PDF-ready format plus the fastest conversion path. Below the document are three one-step methods (browser print-to-PDF is the cleanest and takes ~10 seconds).

I've also made one structural change from the earlier version: I replaced the 6-column tables with structured task blocks. Six columns do not survive PDF pagination — they overflow or shrink to unreadable. Task blocks render identically in every converter and are far easier to check off. Every field you asked for is preserved.

📄 DOCUMENT START — copy everything below this line

LearnByRequest — Implementation Roadmap & Product Optimization Dossier

Version: 2.0
Document type: Implementation roadmap, product scrutiny, and delivery plan
Product: A request-driven micro-learning platform — users file a ticket describing a skill they want to learn; the creator produces a video and replies with the link; the user is notified.
Status legend: ⏳ Next (immediate) · 📋 Planned · 🔄 In Progress · ✅ Done · 🚫 Blocked
Effort legend: S = under 4 hrs · M = 1 day · L = 2–3 days · XL = 1 week+
Role legend: PM (Product) · UX (Design) · FE (Frontend) · BE (Backend) · QA · OPS (DevOps) · LEG (Legal/Ops) · CRT (Creator Ops) · GRW (Growth) · SUP (Support)

How to read a task block: every task carries its own rationale, outcome, dependencies, tooling, acceptance criteria, and responsible role. A task is only "done" when its acceptance criteria are met — not when the code is written.

PART A — Expert Scrutiny & Product Optimization

A1. Core diagnosis

The product loop is Request → Fulfill → Notify → Learn → Retain. It has two structural failure points:

1. Request quality. Vague requests produce unusable videos. A user who asks for "Python" gets a 40-minute ramble they never finish. A user who asks "how do I merge two CSVs in pandas without losing headers" gets a video they complete. Request structure is the single highest-leverage product decision.

2. Notification reliability. If the reply email lands in spam, the loop silently breaks and the user never returns. The notification is not a feature — it is the retention mechanism.

A2. Hidden existential risk: creator bottleneck

You are the sole fulfillment node. Unbounded free requests will outrun your filming capacity within weeks, the queue will silently rot, and trust will collapse. Three mitigations are mandatory, not optional:

· Content reuse. A fulfilled public video is a permanent asset that can answer future similar requests at zero marginal cost. This is the only thing that makes the economics work.
· Queue transparency. Users tolerate waiting only when they can see the queue moving. Expose status.
· Capacity-aware intake. Cap accepted tickets to weekly filming capacity; overflow goes to waitlist with an honest estimate.

A3. The tiering model (approved)

Three tiers, one request flow:

Tier What the user buys Video fate Price
Community A voice in what gets filmed Public, reusable forever Free
Priority Speed — jumps the queue Public, reusable forever Small flat fee
Private Relevance + confidentiality — video built around their files/project Never published; access-restricted Paid, fixed tiers

Why this structure: it gives a clean upsell ladder, gives public requests a reason to stay public (they paid for speed, not secrecy), and makes "why can't I see that video?" answerable.

Critical scoping decision — "private" must mean one thing: bespoke video of the user's actual project/files, not coaching and not merely an unlisted generic video. Paying for privacy on a generic skill is incoherent; coaching destroys filming throughput. State this on the pricing page in one sentence to prevent refund disputes.

A4. The trap nobody sees: unlisted YouTube is not private

An unlisted YouTube link is viewable by anyone holding the URL. If a customer pays for exclusivity, unlisted YouTube is a refund and a reputation hit waiting to happen. Acceptable for free tier; not acceptable once money changes hands without explicit disclosure. Paid private delivery requires signed-URL hosting (Mux, Cloudflare Stream) or Vimeo password/domain privacy.

A5. Risk register

Risk Impact Mitigation Phase
Creator queue overload Trust collapse Capacity cap + waitlist + content reuse 3.4, 5.4
Email lands in spam Loop breaks silently Auth'd domain, fallback in-app bell, bounce logging 3.3, 3.6
Vague requests Wasted filming hours Structured form + clarification step 2.3, 3.2

Duplicate requests Wasted effort Public board + upvotes + library search 2.2, 5.2
Scope creep on paid work Margin death Scope lock at quote/delivery + "new scope = new ticket" 7.2, 7.3
Prohibited content (medical/legal/hacking) Legal liability Moderation gate + prohibited list 2.8
Chargebacks on custom digital goods Revenue loss Delivery confirmation + revision window + clear refund terms 7.3, 7.4
Private video leakage Refund + trust Signed URLs, not unlisted links 7.5
Sales tax / VAT exposure Compliance Stripe Tax 7.7
Legal entity/data exposure Liability Privacy, terms, confidentiality clauses 6.6, 7.6

A6. Five decisions blocking final scope

1. Is "private" bespoke video (recommended) or video + coaching?
2. Fixed price tiers (recommended) or per-request quoting?
3. May private tutorials be re-filmed as public videos later? (Recommended: yes, technique only, never user data — stated in terms.)
4. Launch country and currency? Determines tax and refund-law specifics.
5. Prohibited skill categories list? (medical, legal, financial, hacking, adult, academic-dishonesty…)

PART B — Implementation Roadmap

PHASE 0 — Discovery, Policy & Validation

Phase goal: prove demand exists and lock the rules that shape the UI, before any application code.
Phase status: ⏳ Next
Phase exit criteria: 3+ interviews confirm the pain; minimum-viable-request template frozen; policies drafted; waitlist live.

TASK 0.1 — Author three user personas

· Status: ⏳ Next
· Why: Designing for "learners" is designing for nobody; motivations determine request wording and onboarding copy.
· Achieves: Shared vocabulary for all downstream UX and copy.
· Connects to: 0.2, 2.3, 6.1
· Tools: Notion / Google Docs
· Acceptance: Three named personas with a triggering situation, a blocking moment, and a success definition.
· Role: PM · Effort: S

TASK 0.2 — Run 5–7 problem interviews (15 min each)

· Status: ⏳ Next
· Why: Validates that "I want a video for my exact situation" is a real pain with real urgency.
· Achieves: Go/no-go signal, plus verbatim user language for landing page and form.
· Connects to: 0.5, 2.3, 6.4
· Tools: Calendly, Zoom, a fixed 6-question script
· Acceptance: Written notes for each; one paragraph stating what surprised you; explicit go/no-go.
· Role: PM · Effort: M

TASK 0.3 — Prototype the request form and reply email

· Status: ⏳ Next
· Why: The two surfaces the entire loop depends on deserve design before engineering.
· Achieves: Friction feedback and notification clarity before build cost is incurred.
· Connects to: 2.3, 3.3
· Tools: Figma (or paper + photos)
· Acceptance: Clickable form (5 fields max) and a rendered email mock reviewed by 3 people.
· Role: UX · Effort: M

TASK 0.4 — Freeze the "minimum viable request" template

· Status: ⏳ Next
· Why: Structured input is the highest-leverage quality control in the product.
· Achieves: Filmable requests; fewer clarification round-trips.
· Connects to: 1.3, 2.3, 3.2
· Fields: Title · What exactly do you want to be able to do when this is done? · Current level (dropdown: none / some / confident) · What have you already tried? · Deadline (optional)
· Acceptance: Template written and used in the prototype; each field has a one-line helper text explaining why it's asked.
· Role: PM + UX · Effort: S

TASK 0.5 — Launch waitlist landing page with email capture

· Status: ⏳ Next
· Why: Quantifies demand and builds the beta audience before launch.

· Achieves: Email list for Phase 6.1 invitations.
· Connects to: 6.1, 6.4
· Tools: Carrd / Framer / single Next.js page + Supabase table or ConvertKit
· Acceptance: Page live, ≤1 form field, submission confirmed to a spreadsheet/DB; double opt-in if list > 50.
· Role: GRW + FE · Effort: S

TASK 0.6 — Draft policies in plain language (pre-UI)

· Status: ⏳ Next
· Why: These policies determine UI states; writing them first prevents rework. Also front-loads the most commonly skipped launch blocker.
· Achieves: Refund policy, scope-lock rule, prohibited-content list, confidentiality handling, deletion-on-request, content ownership.
· Connects to: 2.8, 6.6, 7.2, 7.3, 7.6
· Tools: Google Docs → later rendered as site pages
· Acceptance: Each policy ≤1 page, readable by a non-lawyer, reviewed by a lawyer before paid launch.
· Role: LEG + PM · Effort: M

PHASE 1 — Technical Foundation & Data Model

Phase goal: a secure, migration-ready schema that supports free, priority, and private tiers from day one — so monetization is an addition, not a rewrite.
Phase status: ⏳ Next
Phase exit criteria: RLS tests pass; schema supports all three tiers; seed data loads.

Stack decision (documented): Next.js (App Router) + Supabase (Postgres, Auth, RLS, Realtime, Storage). Rationale: one backend for auth/DB/realtime/storage, generous free tier, and a direct reference implementation exists (surveyjs/supabase-helpdesk) for threaded tickets, statuses, and notification triggers.

TASK 1.1 — Provision Supabase project (or equivalent)

· Status: ⏳ Next
· Why: Single backend eliminates integration surface area.
· Achieves: Dev and prod environments ready.
· Connects to: 1.2–1.9
· Acceptance: Two projects (dev, prod); connection strings stored in env vault; local dev via Supabase CLI.

· Role: OPS · Effort: S

TASK 1.2 — Migrate profiles table

· Status: 📋
· Why: Auth gives identity; profiles carry app-specific roles and billing linkage.
· Achieves: Distinguishes requester, creator/admin, and future moderator.
· Connects to: 1.3, 2.1, 7.1
· Fields: id (FK auth.users), display_name, email, role (user/admin), stripe_customer_id (nullable)
· Acceptance: Row auto-created on signup via trigger; RLS: user reads/updates own row only.
· Role: BE · Effort: S

TASK 1.3 — Migrate tickets table (tier-aware)

· Status: 📋
· Why: Core entity; the status enum drives the entire UI and admin queue.
· Achieves: Every request has a trackable lifecycle and an economic tier.
· Connects to: 1.4, 1.6, 2.2, 2.3, 3.1
· Fields: id (UUID), ticket_number (sequential), title, description, skill_category, status (submitted/accepted/in_production/fulfilled/waitlist/declined), visibility (public/private), tier (community/priority/private), price_cents (nullable), payment_status (none/pending/paid/refunded), upvote_count (int, default 0), created_by, created_at, updated_at
· Acceptance: Migration runs clean; indexes on status, visibility, upvote_count, created_by; enum values enforced by DB type.
· Role: BE · Effort: M

TASK 1.4 — Migrate ticket_responses table

· Status: 📋
· Why: Threaded replies separate the public answer from internal notes — essential once volume grows.
· Achieves: Admin replies with video; users see only external responses.
· Connects to: 3.1, 3.2, 3.4
· Fields: id, ticket_id (FK), message, type (external/internal), video_url (nullable), video_source (youtube/mux/vimeo, nullable), created_by, created_at
· Acceptance: FK cascade delete; index on ticket_id; check constraint that type='external' may carry video_url but type='internal' may not be visible to users (enforced in RLS).
· Role: BE · Effort: S

TASK 1.5 — Migrate upvotes table

· Status: 📋
· Why: Prevents double-voting and produces a defensible demand signal.
· Achieves: Accurate upvote counts driving the creator queue.
· Connects to: 2.4, 3.1
· Fields: id, ticket_id, user_id, created_at; unique(ticket_id,user_id)
· Acceptance: Unique constraint enforced; counter updated via trigger, not client.
· Role: BE · Effort: S

TASK 1.6 — Migrate entitlements + payments tables (scaffold now)

· Status: 📋
· Why: Adding billing tables to a live schema is painful; empty tables cost nothing.
· Achieves: Monetization becomes a feature flag, not a migration project.
· Connects to: 7.1, 7.4, 7.5
· Fields — entitlements: id, user_id, ticket_id, access_type (public/private), granted_at, revoked_at
· Fields — payments: id, ticket_id, user_id, stripe_session_id, stripe_payment_intent_id, amount, currency, status, created_at
· Acceptance: Tables exist, RLS on, no UI yet; documented in schema appendix.
· Role: BE · Effort: S

TASK 1.7 — Write and test Row-Level Security policies

· Status: 📋
· Why: Security by default; a single missing policy leaks private requests or payments.
· Achieves: Safe public board and genuinely private requests.
· Connects to: 2.2, 2.5, 4.1
· Policies: users SELECT public tickets; INSERT own tickets; SELECT own private tickets; SELECT external responses on own or public tickets; admin bypass via role claim
· Acceptance: A test suite (pgTAP or scripted SQL) proving a second user cannot read another's private ticket or internal note. This is a hard gate — no UI work before it passes.
· Role: BE + QA · Effort: M

TASK 1.8 — Seed development data

· Status: 📋
· Why: Empty databases hide layout and pagination bugs.

· Achieves: Realistic dev/test fixtures across all tiers and statuses.
· Connects to: 2.2, 3.1
· Acceptance: Seed script creates ~25 tickets, ~40 responses, mixed statuses/tiers, 3 users, 1 admin.
· Role: BE · Effort: S

TASK 1.9 — Set up CI: lint, typecheck, migration dry-run

· Status: 📋
· Why: Schema drift is the most common cause of "works locally, breaks in prod."
· Achieves: Every PR proves migrations apply cleanly.
· Connects to: 6.3
· Tools: GitHub Actions, Supabase CLI
· Acceptance: PR blocked on red CI; migration dry-run against a throwaway DB.
· Role: OPS · Effort: M

PHASE 2 — Core Request Lifecycle (Requester Experience)

Phase goal: a user can discover, sign in, file a structured request, upvote, and track it.
Phase status: 📋
Phase exit criteria: a stranger can submit a request end-to-end unaided on mobile.

TASK 2.1 — Build auth UI (magic link + Google OAuth)

· Status: 📋
· Why: Passwordless removes the #1 signup drop-off; OAuth covers the rest.
· Achieves: Frictionless identity without password support burden.
· Connects to: 2.3, 4.1
· Tools: Supabase Auth, Resend (for magic-link delivery)
· Acceptance: Sign in, sign out, session persists across refresh; auth errors are human-readable; email link works on mobile.
· Role: FE + BE · Effort: M

TASK 2.2 — Build /requests public board

· Status: 📋
· Why: Social proof; shows the queue is alive; makes duplicates visible before they're filed.
· Achieves: Browseable, upvote-sortable board; reduces duplicate filming.

· Connects to: 2.4, 4.4
· Spec: public + open statuses only; sort by upvote_count DESC, created_at DESC; pagination 20/page; server-rendered for SEO
· Acceptance: Read-only for signed-out visitors; ≤1.5s LCP on 4G; pagination works.
· Role: FE · Effort: M

TASK 2.3 — Build /requests/new form (the core action)

· Status: 📋
· Why: This is the product. Every field is deliberate.
· Achieves: Structured, filmable requests.
· Connects to: 1.3, 3.2
· Spec: fields from 0.4; client + server validation; helper text per field; visibility toggle (public/private); tier selector defaulting to community
· Acceptance: Submits in <30s on mobile; validation messages explain how to fix; successful submit redirects to ticket detail with a "what happens next" panel.
· Role: FE + BE · Effort: L

TASK 2.4 — Implement upvote toggle

· Status: 📋
· Why: Demand signal without forcing new tickets; gives signed-out users a reason to sign up.
· Achieves: Prioritization data for the creator queue.
· Connects to: 3.1, 4.4
· Spec: optimistic UI; server action toggles row; counter via trigger; signed-out click prompts auth and preserves intent
· Acceptance: Double-click cannot double-count; count reflects DB truth on reload.
· Role: FE + BE · Effort: S

TASK 2.5 — Build /requests/[id] ticket detail

· Status: 📋
· Why: Single source of truth for requester and public viewers.
· Achieves: Status transparency; video playback; the payoff moment.
· Connects to: 3.2, 4.3
· Spec: title, description, status badge, upvote count, external responses threaded; video renders as embed (YouTube) or signed player (Mux/Vimeo); internal notes never rendered
· Acceptance: Private ticket returns 404 to non-owners (verified by test); video plays inline on mobile.
· Role: FE · Effort: M

TASK 2.6 — Build empty and error states

· Status: 📋
· Why: Blank screens read as broken; first impressions are load-bearing.
· Achieves: Polished feel at zero volume.
· Connects to: 6.3
· Spec: no tickets yet · request in queue · request declined (with reason) · video not yet available · 404/500
· Acceptance: Every list and detail route has a designed empty state with one clear next action.
· Role: UX + FE · Effort: S

TASK 2.7 — Route private/priority intent to the right flow

· Status: 📋
· Why: Private requests need scoping and payment; public ones must not be blocked by it.
· Achieves: One form, three tiers, no dead ends.
· Connects to: 2.8, 7.2
· Spec: "Is this specific to your private files/project?" → yes sets tier='private', reveals scope-note field, shows pricing and next steps; no → community (or priority upsell)
· Acceptance: Tier is set correctly server-side (not trusted from client); private submissions land in a pending state, not the public board.
· Role: FE + BE · Effort: M

TASK 2.8 — Build moderation gate

· Status: 📋
· Why: Users will request medical, legal, financial, or hacking "skills." That is liability, not taste.
· Achieves: Blocked categories never reach filming; you have a paper trail.
· Connects to: 0.6, 3.1
· Spec: prohibited-category checkbox at submit; private requests enter pending_review; admin can decline with a templated reason
· Acceptance: Declined requests notify the user with the reason and a link to the prohibited list; no bypass path exists.
· Role: BE + LEG · Effort: M

PHASE 3 — Fulfillment & Notification Engine (Creator Loop)

Phase goal: you can move a ticket through production and deliver a video that reliably notifies the requester.
Phase status: 📋
Phase exit criteria: an end-to-end test proves reply → email delivered → bell increments → user lands on ticket.

TASK 3.1 — Build /admin/queue cockpit

· Status: 📋
· Why: Filming without a queue view is chaos; priority must be explicit, not remembered.
· Achieves: A single daily operational screen.
· Connects to: 3.2, 5.3, 5.4
· Spec: role-gated; sort by manual priority flag, then upvote_count DESC, then created_at ASC; inline status transitions; filters by tier/status/category; shows payment status for paid tiers
· Acceptance: Non-admin gets 403 (tested); status change persists and updates the requester's view in realtime.
· Role: FE + BE · Effort: L

TASK 3.2 — Build the reply composer

· Status: 📋
· Why: This is the fulfillment action; friction here costs you hours every week.
· Achieves: Video link attached to the thread in under 60 seconds.
· Connects to: 3.3, 3.4, 5.1
· Spec: message textarea; optional video_url with server-side validation (youtube.com / youtu.be / mux / vimeo); draft save; "reply with existing video" picker (links to 5.1)
· Acceptance: Invalid URLs rejected with a clear message; successful submit inserts type='external'; internal-note mode available and provably invisible to users.
· Role: FE + BE · Effort: M

TASK 3.3 — Configure transactional email + template

· Status: 📋
· Why: Email is the primary return trigger; in-app alone cannot bring a user back.
· Achieves: "Your video is ready" delivered reliably.
· Connects to: 3.4, 3.6
· Tools: Resend or Postmark
· Spec: verify sending domain (SPF + DKIM + DMARC); template contains ticket title, creator message,

prominent video CTA, link back to ticket, unsubscribe
· Acceptance: Inbox placement confirmed in Gmail, Outlook, and one mobile client; template renders without images.
· Role: OPS + BE · Effort: M

TASK 3.4 — Implement notification trigger

· Status: 📋
· Why: Automates the critical path; a manual step will eventually be forgotten.
· Achieves: Instant, reliable reply notification.
· Connects to: 3.3, 3.5
· Spec: server-side trigger on ticket_responses insert where type='external'; idempotency key per response ID; retry with backoff
· Acceptance: Integration test proves exactly one email per external reply, even on retry.
· Role: BE · Effort: M

TASK 3.5 — Build in-app notification bell

· Status: 📋
· Why: Fallback for missed email and for active users.
· Achieves: Second notification channel.
· Connects to: 4.1, 4.2
· Acceptance: Unread count accurate; marks read on view; survives refresh.
· Role: FE · Effort: S

TASK 3.6 — Add delivery logging and bounce handling

· Status: 📋
· Why: You must know when a notification fails, or the loop breaks invisibly.
· Achieves: Observability on the most critical action.
· Connects to: 6.3
· Spec: notification_logs table (response_id, provider, status, error, created_at); provider webhook ingestion; admin banner on failures
· Acceptance: A forced bounce appears in logs and in the admin view within 60s.
· Role: BE + OPS · Effort: M

PHASE 4 — Retention & Experience Layer

Phase goal: users return, browse, and give feedback; public fulfilled tickets become an acquisition surface.
Phase status: 📋

TASK 4.1 — Build /dashboard

· Status: 📋 · Why: Users need one home for all their requests. · Achieves: Return visits. · Connects to: 4.2
· Acceptance: Lists own tickets with status badges and "new reply" indicators; empty state links to /requests/new. · Role: FE · Effort: M

TASK 4.2 — Realtime updates on own tickets

· Status: 📋 · Why: Live feedback makes the app feel alive without refresh. · Achieves: Immediate bell/dashboard updates. · Connects to: 3.5
· Acceptance: Reply inserted in DB updates the bell in a second tab within 3s. · Role: FE + BE · Effort: M

TASK 4.3 — "Did this help?" feedback prompt

· Status: 📋 · Why: Closes the quality loop; tells you which requests to screen harder and which video style works. · Achieves: Feedback data on fulfillment quality. · Connects to: 6.2
· Spec: thumbs up/down + optional comment, shown once per response · Acceptance: Stored per response; visible in admin; never blocks playback. · Role: FE + BE · Effort: S

TASK 4.4 — "Similar open requests" suggestions

· Status: 📋 · Why: Keeps users browsing and upvoting after submitting. · Achieves: Session depth + demand signal. · Connects to: 2.4
· Acceptance: Three related tickets shown by skill_category overlap; excludes own. · Role: BE · Effort: S

TASK 4.5 — Public fulfilled ticket pages (SEO)

· Status: 📋 · Why: Each video becomes a long-tail landing page — compounding organic growth. · Achieves: Search traffic. · Connects to: 6.4

· Spec: only visibility='public' + status='fulfilled'; slug from title; structured data (VideoObject); no user PII
· Acceptance: Pages indexable, sitemap generated, Lighthouse SEO ≥ 95. · Role: FE + GRW · Effort: M

PHASE 5 — Creator Operations & Content Reuse

Phase goal: fulfillment scales without proportionally more filming.
Phase status: 📋 (trigger: queue > 10 active tickets)

TASK 5.1 — Add "reply with existing video"

· Status: 📋 · Why: The single biggest creator-load reduction available. · Achieves: Zero-filming fulfillment for repeat requests. · Connects to: 5.2, 3.2
· Acceptance: Picker searches fulfilled videos; reply marks ticket fulfilled (existing); user sees no difference. · Role: FE + BE · Effort: M

TASK 5.2 — Build /admin/library

· Status: 📋 · Why: Reuse requires findability. · Achieves: Searchable content asset inventory. · Connects to: 5.1
· Acceptance: Search by title, category, tag; shows usage count per video. · Role: FE · Effort: M

TASK 5.3 — Internal notes thread

· Status: 📋 · Why: Operational memory; smoother handoff if you add a collaborator. · Achieves: Private context per ticket. · Connects to: 3.1, 1.7
· Acceptance: Notes never appear in any user-facing query (tested). · Role: BE · Effort: S

TASK 5.4 — Capacity-aware waitlist

· Status: 📋 · Why: Silent backlog destroys trust faster than an honest wait. · Achieves: Sustainable pace with honest expectations. · Connects to: 3.1, 3.3
· Spec: configurable weekly capacity; overflow auto-sets waitlist + emailed estimate; auto-promote when capacity frees
· Acceptance: Setting capacity to N moves the (N+1)th request to waitlist with an email. · Role: BE +

CRT · Effort: M

TASK 5.5 — Weekly creator digest

· Status: 📋 · Why: Accountability and demand-trend awareness without dashboard-checking. · Achieves: Data-driven filming priority. · Connects to: 6.2
· Acceptance: Monday email: new requests, top upvoted, fulfilled count, average time-to-fulfillment. · Role: BE · Effort: S

PHASE 6 — Launch, Measurement & Iteration

Phase goal: ship to real users, measure the loop, fix what's broken.
Phase status: 📋

TASK 6.1 — Private beta with first 20 users

· Status: 📋 · Why: Controlled environment to catch bugs and observe real behavior. · Achieves: Feedback, testimonials, and first filled library. · Connects to: 0.5, 6.2
· Acceptance: 20 invitees, 10+ requests filed, 5+ fulfilled, 3 recorded interviews. · Role: PM + SUP · Effort: L

TASK 6.2 — Instrument the funnel

· Status: 📋 · Why: You cannot optimize what you do not measure. · Achieves: Baseline metrics. · Connects to: 6.5
· Metrics: request submission rate · time-to-fulfillment · email open & click rate · video watch start · 7-day return rate · upvotes per request
· Tools: PostHog or Plausible
· Acceptance: Dashboard shows all metrics; no PII in event payloads. · Role: BE + PM · Effort: M

TASK 6.3 — Error monitoring and alerting

· Status: 📋 · Why: Silent failures kill trust faster than loud ones. · Achieves: Alerting on critical-path failures. · Connects to: 3.6
· Acceptance: Sentry (or equivalent) captures API, email, and RLS denials; alert routes to your phone. ·

Role: OPS · Effort: S

TASK 6.4 — SEO baseline

· Status: 📋 · Why: Compounding acquisition channel. · Achieves: Discoverability beyond direct traffic. · Connects to: 4.5
· Acceptance: Meta tags, OG images, sitemap, robots.txt, verified in Search Console. · Role: GRW · Effort: S

TASK 6.5 — Define iteration cadence

· Status: 📋 · Why: Prevents post-launch stagnation. · Achieves: Continuous improvement habit. · Connects to: 6.2
· Acceptance: Calendar block: weekly metrics review, bi-weekly shipped improvement, documented in a changelog. · Role: PM · Effort: S

TASK 6.6 — Legal hygiene before public launch

· Status: 📋 · Why: Required, and protects you specifically on user-submitted content and educational disclaimers. · Achieves: Compliance and trust. · Connects to: 0.6
· Acceptance: Privacy policy, ToS, content disclaimer, prohibited-content list all live and linked in footer. · Role: LEG · Effort: M

PHASE 7 — Monetization (Priority & Private Tiers)

Phase goal: accept money safely and deliver genuinely private video.
Phase status: 📋 — Hard gate: do not start until Phase 6 metrics show a healthy free loop.
Rationale for gating: payments add ~30–40% more engineering and a permanent support burden; the free tier first proves demand and builds the library you'll sell against.

TASK 7.1 — Stripe Checkout + entitlement grant

· Status: 📋 · Why: Never build custom payment UI. · Achieves: Paid tier purchase. · Connects to: 1.6
· Stripe events to handle: checkout.session.completed, payment_intent.succeeded, charge.refunded, charge.dispute.created

· Acceptance: Webhook signature verified; idempotent (replay grants once); entitlement row created; failure path notifies admin. · Role: BE · Effort: L

TASK 7.2 — Fixed-price tiers (no quoting)

· Status: 📋 · Why: Quoting adds a whole state machine and support load; fixed tiers ship faster. · Achieves: Predictable purchase. · Connects to: 2.7, 7.3
· Acceptance: Two products live (priority, private-standard); price and scope text match the policy page exactly. · Role: BE + PM · Effort: M

TASK 7.3 — Delivery confirmation + 7-day revision window

· Status: 📋 · Why: Your primary chargeback defense on custom digital goods. · Achieves: Documented satisfaction or dispute path. · Connects to: 7.4
· State machine: requested → quoted/accepted → paid → in_production → delivered → (satisfied | revision_requested | disputed) → closed
· Acceptance: User must actively confirm; window enforced server-side; one revision included, second is a new ticket. · Role: BE + UX · Effort: L

TASK 7.4 — Refund & dispute handling

· Status: 📋 · Why: Must be automatic, not discovered by email. · Achieves: Correct entitlement state and revenue reporting. · Connects to: 7.1
· Acceptance: Refund webhook revokes access and notifies both parties; dispute alerts immediately. · Role: BE · Effort: M

TASK 7.5 — Signed-URL video hosting for private deliveries

· Status: 📋 · Why: Unlisted YouTube is not private; paying customers deserve real access control. · Achieves: Leak-resistant private video. · Connects to: 3.2
· Tools: Mux or Cloudflare Stream (signed playback tokens tied to session)
· Acceptance: Direct URL sharing fails outside an authenticated session; token expiry enforced. · Role: BE + OPS · Effort: L

TASK 7.6 — Publish commercial terms

· Status: 📋 · Why: Scope, ownership, confidentiality, and refund terms must precede taking money. · Achieves: Dispute resistance and legal clarity. · Connects to: 0.6
· Must state: what "private" means · scope lock · additional scope = new ticket · who owns the recording · deletion on request · re-film rights (technique only) · refund policy
· Acceptance: Lawyer-reviewed; linked at checkout. · Role: LEG · Effort: M

TASK 7.7 — Stripe Tax configuration

· Status: 📋 · Why: Selling digital services cross-border triggers VAT/sales-tax obligations. · Achieves: Compliant collection. · Connects to: 7.2
· Acceptance: Tax registrations configured for target jurisdictions; invoices show tax lines. · Role: OPS + LEG · Effort: M

PART C — Dependency Map

```
0.4 ─┬─> 1.3 ─┬─> 2.3 ─> 2.5 ─> 3.2 ─> 3.4 ─> 6.2
     │        ├─> 2.2 ─> 2.4 ─> 3.1
0.6 ─┴─> 2.8   └─> 1.7 (HARD GATE) ─> 2.5, 4.1

3.4 ─> 3.3 ─> 3.6 ─> 6.3
2.5 ─> 4.1 ─> 4.2
5.1 ─> 5.2  (gate: queue > 10)

PHASE 6 metrics healthy ──> PHASE 7 (gate)
7.1 ─> 7.3 ─> 7.4
7.5 (independent, required before private delivery)
```

Three hard gates:

1. 1.7 RLS tests must pass before any user-facing read/write UI.
2. Phase 6 metrics healthy before Phase 7 begins.
3. 7.5 signed-URL hosting + 7.6 terms before accepting payment for private tutorials.

PART D — Immediate Next Actions (48 hours)

1. 0.1–0.4 — personas, interviews, prototype, frozen request template.

2. 0.6 — draft policies (they inform the form UI).
3. 1.1 — provision the Supabase project.
4. Answer the five decisions in A6 — they determine whether Phase 7 has one price tier or three.

Do not write application code until at least three interviews confirm the pain. If they do, scaffold the schema (1.3, 1.4, 1.6) and write the RLS tests (1.7) before touching UI. That gives you the full skeleton — free, priority, and private — before you invest in polish.

📄 DOCUMENT END

How to turn this into a PDF (pick one, ~10 seconds)

Option 1 — Browser (cleanest, no install):

1. Copy everything between DOCUMENT START and DOCUMENT END.
2. Paste into dillinger.io or stackedit.io (or any Markdown editor).
3. Ctrl/Cmd + P → Save as PDF → set margins to "Default," enable "Background graphics."

Option 2 — VS Code (best formatting control):

1. Save as roadmap.md.
2. Install the Markdown PDF extension.
3. Ctrl/Cmd + Shift + P → "Markdown PDF: Export (pdf)".

Option 3 — CLI (if you want it in your repo):

```bash
npx md-to-pdf roadmap.md
```

Option 4 — Google Docs: paste into a new Doc, then File → Download → PDF. Tables and headings carry over; check the task blocks render with their bullet indentation intact.

Two things worth flagging before you commit:

1. The document is intentionally decision-gated. Part A6 has five unanswered questions. Phases 0–6 are safe to build regardless of the answers; Phase 7's shape depends entirely on them. Answer those five and I'll rewrite Phase 7 with per-task granularity, exact Stripe event payloads, and the private-delivery state machine broken into individual checklist items.

2. The highest-risk assumption is still untested. Everything here assumes people will file a structured request rather than just searching YouTube. That's what Tasks 0.2 and 0.3 exist to disprove cheaply. Spend the 48 hours there before the 6 weeks of building.
3.
Verdict: Yes — but you've just opened a second product. Handle it deliberately.

Your instinct is correct, and it solves the single biggest economic flaw in the original model: a one-off video for one user is a cost center; a reusable video is an asset. Letting fulfilled community videos serve future users turns your creator hours into compounding inventory. That is the right move.

But "private tutorial with payments and confirmation" is not a small feature — it is a separate product with a separate state machine, separate legal exposure, and separate ops burden. If you bolt it on carelessly, it will eat your launch. Here is the scrutiny.

---

1. Define "private" before you build anything

There are three very different things people mean by "private tutorial," and they have wildly different prices and effort:

Type What it means Effort Price band
A. Unlisted video Same generic video, but only you can access it Near-zero (same filming) Low
B. Bespoke video Video uses your files, your project, your data High (screen recording of their context) Mid–High
C. Coaching Video + 1:1 call + follow-up Q&A Very high (calendar, support) High

My recommendation: launch with B only. Type A is pointless (why pay for privacy on a generic skill?). Type C turns you into a consultant and will destroy your filming throughput. Type B is the honest, defensible product: "Send me your actual file/project and I'll make a video walking through your exact situation." That is genuinely worth money and is still bounded work.

Say this out loud in your pricing page. Vagueness here creates refund disputes.

---

2. The tiering model that makes this coherent

Do not have two products. Have one request flow with three tiers:

1. Community request (free) — public board, upvote-driven, video becomes part of the public library, reusable forever.
2. Priority request (small flat fee) — jumps the queue, but the video is still public and reusable. You are selling speed, not exclusivity.
3. Private tutorial (paid, scoped) — video built around the user's specific files/context, never published,

access restricted to them. You are selling relevance and confidentiality.

This gives you:

· A clean upsell ladder
· A reason for public requests to stay public (they paid for speed, not secrecy)
· A clear answer to "why can't I see that video?" → it was a private tutorial

Decide this now: if a private tutorial later turns out to be generically useful, can you re-film a public version? My answer: yes, but only as a new video with no user data — and state this in your terms so nobody feels betrayed.

---

3. The payment + confirmation state machine

You said "payments and confirmation for stuff like that" — that phrase hides an entire engineering surface. Here is what it actually requires:

```
requested → quoted → quote_accepted → paid → in_production
→ delivered → (satisfied | revision_requested | disputed) → closed
```

Confirmation happens twice, not once:

· Scope confirmation (before you film): you review the request, send a fixed quote or agree scope, user accepts. This protects you from "that's not what I wanted."
· Delivery confirmation (after you deliver): user marks satisfied, or has a 7-day window to request one revision or open a dispute. This protects you from chargebacks.

Technical requirements this forces:

· Stripe Checkout (do not build custom payment UI) + webhook handler that grants entitlement on checkout.session.completed
· Idempotency on webhooks (Stripe retries; you must not double-grant)
· An entitlements table: user_id, ticket_id, access_type, granted_at, revoked_at
· Refund/dispute flow: webhook on charge.refunded / charge.dispute.created flips entitlement and notifies you
· A written refund policy in your terms (digital custom goods — your jurisdiction's distance-selling rules may give users withdrawal rights; check this)

Do not do quote-based pricing at MVP. Fixed tiers ("Private tutorial – standard $X / with your files $Y") remove an entire state machine and a support burden. Add quoting in v2 if demand justifies it.

---

4. Secure hosting for private videos (the trap nobody sees)

An unlisted YouTube link is not private — anyone with the URL can view it, and it can leak. If a customer is paying for exclusivity, unlisted YouTube is a refund waiting to happen.

Options ranked for effort:

· MVP-acceptable: unlisted YouTube + explicit terms saying "access is via private link, do not share." Cheap, fast, slightly leaky. Fine for low-stakes skills.
· Recommended once paid: Vimeo with domain/ password privacy, or Mux / Cloudflare Stream with signed playback tokens tied to the user's session. This is real access control and is what "private" should mean if money changes hands.

Flag this as a Phase 7 blocker: do not take payment for private tutorials while hosting on unlisted YouTube without disclosing the limitation.

---

5. Risks you must actively manage

· Scope creep — "while you're at it, can you also…" Kills your margins. Mitigate with a hard scope lock at quote acceptance and a written "additional scope = new ticket" rule.
· Prohibited content — users will request medical, legal, financial, or hacking "skills." Add a moderation gate at submission (checkbox + review) and a written prohibited-categories list. This is liability, not just taste.
· Content ownership — who owns a bespoke video of the user's project? Define it: user owns the recording of their data; you retain the right to reuse technique and generic re-films. Put it in terms.
· Confidentiality — if they send you their codebase or business data, you now hold sensitive material. A one-paragraph handling policy and deletion-on-request clause is cheap insurance.
· Chargebacks — custom digital work is a chargeback magnet. Delivery confirmation + revision window + clear refund terms are your defense.
· Tax — payment means sales tax/VAT depending on buyer location. Use Stripe Tax or accept the manual burden knowingly.

---

6. The sequencing advice (most important part)

Do not build the paid private tier for MVP. Build the free community tier, prove the loop works, get 20–50 fulfilled videos, then layer payments on. Reasons:

· The free tier validates demand and builds the content library you will later reuse and sell against
· Payment adds ~30–40% more engineering and a permanent support burden — you do not want that on

day one
· You will learn what people actually pay for by watching which free requests get the most upvotes and repeat asks

So: Phase 7 becomes "Monetization: paid priority + private tutorials," gated behind Phase 6 metrics showing the free loop is healthy.

---

7. Deltas to the roadmap

Add these to your todo list. Everything else from the prior roadmap stands.

Amend Phase 1 (data model) now, before you migrate:

· tickets gains: tier (community | priority | private), is_public (already have visibility — reuse), price_cents, payment_status (none | pending | paid | refunded)
· New entitlements table (see §3)
· New payments table: stripe_session_id, stripe_payment_intent_id, amount, currency, status, ticket_id, user_id
· profiles gains: stripe_customer_id

Add to Phase 0:

· 0.6 — Write the refund, scope-lock, and prohibited-content policies in plain language before you build the private flow (they inform the UI).

Add Phase 2 tasks:

· 2.7 — Request form: "Is this specific to your private project/data?" → routes to private paid flow with a scope note field
· 2.8 — Moderation gate: prohibited-category checkbox + admin review queue for private requests

New Phase 7 (post-launch): Monetization

· 7.1 — Stripe Checkout + webhook + entitlements grant
· 7.2 — Quote/scope confirmation UI (or fixed-price tiers, preferred for v1)
· 7.3 — Delivery confirmation + 7-day revision window UI
· 7.4 — Refund/dispute webhook handling + admin notification
· 7.5 — Signed-URL video hosting (Mux/Cloudflare Stream) for private deliveries
· 7.6 — Terms, privacy, refund, ownership, and confidentiality pages
· 7.7 — Stripe Tax configuration

---

8. Decisions I need from you before I can tighten the roadmap further

1. Is "private" B (bespoke video of their project) or C (video + coaching)? I assumed B.
2. Fixed price tiers, or per-request quoting? I recommend fixed.
3. Do private tutorials ever get re-filmed as public videos? I recommend yes, with terms.
4. What is your launch country / currency? This determines tax and refund-law specifics.
5. What skill categories are prohibited? (medical, legal, financial, hacking, adult, etc.)

Answer those five and I will rewrite the affected phases with the same granularity as the original roadmap, including the exact schema deltas, the Stripe webhook event list, and the private-delivery state machine as individual todo items.

Before the roadmap, here is the critical scrutiny you asked for. Your core loop is Request → Fulfill → Notify → Learn → Retain. The app lives or dies on two things: request quality (vague requests create unusable videos) and notification reliability (if the user misses the reply, the loop breaks). The biggest hidden risk is creator bottleneck: you are the sole fulfillment node. The roadmap below bakes in guardrails, request structuring, and a creator-ops layer so the system does not collapse under its own demand.

Expert Scrutiny & Optimization

Product & Value Proposition

· The "Point of Need" is specificity, not generic learning. A user who requests "Python" gets a 40-minute ramble. A user who requests "How to merge two CSV files in Python pandas without losing headers" gets a video they actually finish. The request form must force context: current level, exact use case, and what "done" looks like. This is the single highest-leverage UX decision.
· Add lightweight demand signaling. Allow users to upvote existing open requests instead of only creating new ones. This does three things: reduces duplicate tickets, tells you what to film next based on real demand, and makes users feel heard even before a video exists.
· Transparency builds trust. Users tolerate waiting only if they see a status. Expose a simple status tracker: Submitted → Accepted → In Production → Video Ready. Without this, silence feels like abandonment.
· The notification is the product. If the email lands in spam, the user never returns. Multi-channel fallback (in-app bell + email) is non-negotiable for MVP.

Technical Architecture (Recommended)

· Next.js + Supabase is the fastest path to a production-grade MVP. Supabase gives you Postgres, Auth, Row-Level Security, Storage, and Realtime out of the box. The surveyjs/supabase-helpdesk repo is a direct reference implementation for ticket threading, statuses, and in-app/email notifications.
· Ticket schema: mirror proven helpdesk structures. A tickets table (id, title, description, status, priority,

created_by, created_at) and a ticket_responses table (id, ticket_id, message, type [external/internal], video_url, created_at) covers the entire loop.
· Notification layer: use Resend (or Postmark) for transactional email. Trigger on ticket_responses insert where type = 'external'. Coalesce notifications if you ever add bulk actions, but for MVP, send immediately on reply.
· YouTube link handling: you do not need the YouTube Data API for MVP. You are manually pasting a URL into the reply. Validate the URL format server-side and embed it via iframe or a thumbnail card. If you later want to auto-detect when a video goes live, WebSub push is the correct backbone, not polling. But that is a Phase 6 optimization, not an MVP blocker.

UX & Onboarding

· Do not require account creation to browse. Let users see the request board (read-only) before signing up. Require auth only when they submit or upvote. This reduces friction and lets the value proposition prove itself.
· First-session win: after a user submits their first request, show them exactly what happens next ("You'll get an email the moment your video is ready") and surface 2–3 existing requests they can upvote. This converts a one-off action into engagement.

Operations & Sustainability

· Creator queue discipline: you need an internal admin view sorted by priority, upvotes, and age. Cap the number of Accepted tickets to what you can realistically film per week. If the queue backs up, switch new requests to Waitlist so expectations stay honest.
· Content reuse: after fulfilling a request, tag the video by skill. Over time, when a new user requests something similar, you can reply with an existing video and mark the ticket Fulfilled (Existing) . This dramatically reduces creator load.

Growth & Legal

· SEO surface: make fulfilled tickets public (with user permission) as indexed pages. Each video becomes a landing page for a long-tail skill query.
· Privacy: allow users to submit requests either publicly (visible on board) or privately (only visible to them and you). Include a simple terms page and a privacy policy before launch.

Ultra-Granular Implementation Roadmap

Legend: ✅ Done | 🔄 In Progress | ⏳ Next (immediate priority) | 📋 Planned (future phase)

Phase 0: Discovery & Validation (Pre-Code)

Status: ⏳ Next

This phase prevents you from building something nobody wants. It is not optional.

Task Status Why (Rationale) Achieves (Outcome) Connects To
0.1 Draft 3 user personas: "career switcher," "freelancer upskilling," "hobbyist with a deadline" ⏳ Forces you to design for real motivations, not abstract "learners" Clear language for landing page and request prompts 0.2, 2.1
0.2 Run 5–7 problem interviews (15 min each) asking what skill they tried to learn recently and where they got stuck ⏳ Validates that "request a video" is a real pain, not just a neat idea Go/no-go signal and exact phrasing users use 0.3, 0.4
0.3 Build a clickable Figma/paper prototype of the request form + reply notification email ⏳ Tests the core loop before writing code Feedback on form friction and notification clarity 0.5, 2.2
0.4 Define the "minimum viable request" template: Title, What do you want to build/do?, Current level (dropdown), Deadline (optional) ⏳ Structured input is the highest-leverage quality control Consistent, filmable requests; fewer back-and-forth clarifications 2.3, 3.3
0.5 Set up a waitlist landing page (Carrd/Framer or simple Next.js page) with email capture ⏳ Builds an audience before launch; validates demand quantitatively Email list for beta invitations 6.1

Phase 1: Technical Foundation & Data Model

Status: ⏳ Next (start immediately after 0.4)

Task Status Why Achieves Connects To
1.1 Create Supabase project (or equivalent Postgres + Auth provider) ⏳ Single backend for DB, auth, storage, and realtime Environment ready for schema 1.2, 1.3
1.2 Design and migrate profiles table: id (FK auth.users), display_name, email, role (user/admin) ⏳ Extends auth with app-specific roles Distinguishes requester from creator/admin 1.4, 2.1
1.3 Create tickets table: id (UUID), ticket_number (sequential), title, description, skill_category, status (submitted/accepted/in_production/fulfilled/waitlist), visibility (public/private), created_by, created_at, upvote_count ⏳ Core entity; status enum drives UI and admin queue Every request has a trackable lifecycle 1.4, 2.2, 4.1
1.4 Create ticket_responses table: id, ticket_id (FK), message, type (external/internal), video_url (nullable), created_by, created_at ⏳ Threaded replies separate public answer from internal notes Admin can reply with video; users see only external 3.1, 3.2
1.5 Create upvotes table: id, ticket_id, user_id, unique constraint ⏳ Prevents double-voting; powers demand signal Accurate upvote counts 2.4, 4.3
1.6 Enable Row-Level Security (RLS): users can SELECT public tickets, INSERT their own tickets, SELECT their own private tickets, SELECT external responses on their tickets ⏳ Security by default; prevents data leaks Safe public board and private requests 2.2, 4.2
1.7 Set up storage bucket for future thumbnail uploads (optional for MVP) 📋 Not needed for link-based replies; defer until video thumbnail customization is required — Phase 5

Phase 2: Core Request Lifecycle (The "Ticket" Experience)

Status: 📋 Planned (starts after Phase 1.6 passes RLS tests)

Task Status Why Achieves Connects To
2.1 Build auth UI: email magic link (passwordless) and/or OAuth (Google) 📋 Lowest-friction auth; magic link avoids password fatigue Users can sign in without remembering credentials 2.2, 4.2
2.2 Build /requests public board: paginated list of status IN ('submitted','accepted','in_production') and visibility = 'public', sorted by upvote_count DESC, created_at DESC 📋 Social proof and demand transparency; users see the queue is alive Browseable board that reduces duplicate requests 2.4, 4.1
2.3 Build /requests/new form with fields from 0.4; on submit, insert ticket with status = 'submitted' and redirect to ticket detail 📋 The core action of the product User successfully files a request 3.1, 4.1
2.4 Implement upvote button on each public open ticket; server action toggles row in upvotes and increments upvote_count 📋 Demand signaling without requiring users to create new tickets Prioritization data for creator queue 3.4, 4.3
2.5 Build /requests/[id] detail page: show title, description, status badge, upvote count, and external responses (video embed if video_url exists) 📋 Single source of truth for the requester and public viewers User can track their request and watch the reply 3.2, 4.1
2.6 Add empty states: "No requests yet — be the first" and "Your request is in the queue" 📋 Reduces bounce on blank screens; sets expectations Polished first impression 6.3

Phase 3: Fulfillment & Notification Engine (The Creator Loop)

Status: 📋 Planned

Task Status Why Achieves Connects To
3.1 Build /admin/queue view (role-gated): list all tickets sorted by priority (manual flag), upvote_count DESC, created_at ASC; allow status transitions (accepted, in_production, fulfilled, waitlist) 📋 Creator needs a single operational cockpit; prevents chaos Efficient daily filming workflow 3.3, 4.3
3.2 Build reply composer in /admin/queue/[id]: textarea for message, optional video_url input with URL validation (must be youtube.com or youtu.be), submit inserts ticket_responses row with type = 'external' 📋 This is the "fulfillment" action that closes the loop Video link is attached to the ticket thread 3.3, 4.1
3.3 Configure transactional email provider (Resend/Postmark) and create email template: "Your video is ready" with ticket title, creator message, video link button, and link back to ticket 📋 Email is the primary return trigger; in-app alone is insufficient User is notified even when app is closed 3.4, 6.2
3.4 Implement server-side notification trigger: on ticket_responses insert where type = 'external', call email provider with requester's email and template variables 📋 Automates the critical notification without manual sending Reliable, instant reply notification 3.3, 6.3
3.5 Build in-app notification bell: count unread external responses on user's tickets; mark as read on ticket view 📋 Fallback for users who miss the email or have notifications off Second notification

channel 4.2
3.6 Add email delivery error logging (webhook from Resend/Postmark into notification_logs table) 📋 If email bounces, you must know and retry or switch channel Observability for the most critical action 6.3

Phase 4: User Retention & Experience Layer

Status: 📋 Planned (can ship incrementally after Phase 3.4)

Task Status Why Achieves Connects To
4.1 Build /dashboard for requesters: list of their tickets with status badges and reply indicators 📋 Users need a home to see all their requests at a glance Return visits to check status 4.2
4.2 Implement realtime subscription on ticket_responses for the current user's tickets; update bell count and dashboard live 📋 Supabase Realtime makes the app feel alive without refresh Immediate feedback when a reply lands while user is active 3.5
4.3 Add "Request fulfilled?" prompt when a user views a ticket with a new video: thumbs up/down + optional comment 📋 Closes the quality loop; tells you what content actually helps Feedback data for improving video style and request screening 6.3
4.4 Add "Suggested requests" on ticket detail: 3 other open tickets with overlapping skill_category 📋 Keeps users browsing and upvoting after they submit Increased session depth and demand signal 2.4
4.5 Implement public fulfilled ticket pages (SEO-friendly slugs) for tickets with visibility = 'public' and status = 'fulfilled' 📋 Long-tail organic acquisition; each video becomes a landing page Search traffic and passive user growth 6.4

Phase 5: Creator Operations & Content Reuse

Status: 📋 Planned (Phase 5 starts when queue exceeds ~10 active tickets)

Task Status Why Achieves Connects To
5.1 Add existing_video reply type: when replying, creator can select from a library of past fulfilled videos instead of filming new 📋 Directly reduces creator load; scales fulfillment without more filming Faster replies for common requests 5.2
5.2 Build /admin/library: searchable list of fulfilled tickets with video URLs and skill tags 📋 Makes reuse practical; prevents duplicate filming Content asset inventory 5.1, 3.1
5.3 Add internal_notes field/thread on tickets visible only to admin (separate from external responses) 📋 Creator can note "needs scripting," "already covered in video X," or "too vague, asked follow-up" Operational memory; smoother handoff if you ever add a collaborator 3.1
5.4 Implement waitlist status and auto-email: when queue exceeds your weekly capacity, new requests go to waitlist with an estimated timeline 📋 Manages expectations honestly; prevents silent backlog frustration Sustainable creator pace 3.1, 3.3
5.5 Weekly digest email to creator: "X new requests, Y upvotes, Z fulfilled last week" 📋 Keeps you accountable and aware of demand trends Data-driven filming prioritization 6.3

Phase 6: Launch, Measurement & Iteration

Status: 📋 Planned

Task Status Why Achieves Connects To
6.1 Invite waitlist (Phase 0.5) to private beta; onboard first 20 users manually 📋 Controlled environment to catch bugs and observe real behavior Early feedback and testimonials 6.2
6.2 Instrument analytics: request submission rate, time-to-fulfillment, email open rate, video click-through rate, 7-day return rate 📋 You cannot optimize what you do not measure Baseline metrics for iteration 6.3
6.3 Set up error monitoring (Sentry or Supabase logs) for API routes, email failures, and RLS denials 📋 Silent failures kill trust faster than bugs Alerting on critical path failures 3.6
6.4 Add basic SEO: meta tags, sitemap for public fulfilled tickets, Open Graph images 📋 Compounding organic growth channel Discoverability beyond direct traffic 4.5
6.5 Define iteration cadence: review metrics weekly, ship one UX or retention improvement bi-weekly 📋 Prevents stagnation after launch Continuous improvement loop All phases
6.6 Legal hygiene: publish privacy policy, terms of service, and a simple content disclaimer (videos are educational, not guaranteed outcomes) 📋 Required before public launch; protects you and users Trust and compliance 6.1
6.7 Plan Phase 7 candidates based on data: community creators, paid priority requests, skill-path bundling, mobile push notifications 📋 Roadmap should be data-driven, not speculative Sustainable long-term product direction 6.2

What to do right now (next 48 hours): Execute tasks 0.1–0.4. Do not write a line of application code until at least three problem interviews confirm that "I want a specific video for my exact situation" is a pain people will act on. If the interviews validate it, immediately scaffold the Supabase project (1.1) and migrate the tickets and ticket_responses tables (1.3, 1.4). That gives you the skeleton of the entire product before you invest in UI polish.

The complete implementation roadmap for your learning-request platform

Master project plan · Discovery to launch, monetization and scale

We're going to treat this as a real product build, not just a list of features. The roadmap will cover product strategy, user experience, frontend and backend engineering, database design, security, payments, notifications, video delivery, testing, deployment, operations, marketing, and long-term growth.

The core product principle is:

People tell the platform what they want to learn. The platform helps them find an existing answer or request a new one. You create the tutorial, attach it to the request, and the learner is notified. Public tutorials help everyone; private tutorials are delivered under an agreed paid arrangement.

1. Project status right now

We need to distinguish between decisions we've discussed and implementation work that has actually been completed. We haven't established that a codebase, database, or deployed app exists yet.

Master project tracker

(Planning stage)

Status reflects this conversation only. No development, testing, or deployment is being claimed as completed.

What we've actually accomplished

- [x] Defined the core concept: learning requests handled like tickets.
- [x] Established the public tutorial model, where a completed video can serve many learners.
- [x] Established the private custom tutorial model, with agreed scope and payment.
- [x] Identified the value of an existing tutorial library, request matching, status tracking, notifications and feedback.
- [x] Established the principle of validating demand before overbuilding the platform.

What comes next

- [ ] Finalize the product requirements and boundaries.
- [ ] Validate the most promising first audience.
- [ ] Design the user journeys and data model.
- [ ] Choose the technology stack and external services.
- [ ] Build and test a complete, usable MVP.
- [ ] Introduce paid private requests only after the core request-to-delivery workflow is reliable.

The checklist below is the master backlog. Every task should eventually have an owner, status, priority, dependencies, completion evidence and acceptance criteria.

2. How to use this roadmap

Every task uses four levels of tracking:

- Status: whether it is planned, underway, blocked or verified as complete.
- Why: the problem it solves and the value it creates.
- Tools and connections: the systems, services or people needed.
- Done when: the evidence that proves the task is complete.

Priority key:

(P0) Required before the MVP can safely launch (P1) Important soon after the core workflow (P2) Later enhancement or scale improvement

The roadmap is intentionally granular. You can work through it sequentially, but some tasks within a phase can be done in parallel once their dependencies are satisfied.

3\. The full roadmap, phase by phase

I've also created a working spreadsheet containing 163 individual implementation tasks, each with its own ID, priority, status, rationale, tools and connections, acceptance criteria, dependencies, owner, target date and notes.

Learning Platform — Master Implementation Checklist

Excel workbook · 163 tasks · 15 worksheets sections including project guidance

Download the implementation roadmap (.xlsx)

Use the Master Checklist sheet to track progress. Update a task to “Done (verified)” only when its acceptance criteria have been met.

The workbook is the detailed task register. The following sections explain the implementation logic and the decisions that matter most.

Phase 0 — Product decisions and project setup

P0 · Start here

* Create a project workspace and task tracker.

* Define the product promise and target audience.

* Choose the initial learning niche.

* Document public versus private request rules.

* Define the meaning of private access.

* Establish realistic response and delivery expectations.

* Agree on MVP scope and features to postpone.

* Create a decision log, risk register and definition of done.

* Estimate your weekly capacity for researching, producing and publishing tutorials.

Why: A technically sound app can still fail if the promise, audience, workflow or workload is poorly defined.

Tools: A project tracker such as GitHub Projects, Linear or Trello; a document workspace; a decision log.

Done when: You have a clear product specification, defined boundaries and a realistic plan for fulfilling requests.

Phase 1 — User research and validation

P0

* Interview 8–15 potential learners.

* Collect examples of real questions they couldn't answer satisfactorily.

* Discover where they currently search for tutorials.

* Find out why existing videos don't meet their needs.

* Test their preference for public versus private tutorials.

* Investigate willingness to pay for specialized help.

* Collect 20–50 real requests manually.

* Group repeated requests into topic clusters.

* Test the entire service manually before automating it.

* Establish evidence-based criteria for proceeding.

Why: This tells us what to build, who needs it and which requests are worth fulfilling first.

Tools: Interviews, a simple form, a spreadsheet, YouTube and a manual delivery workflow.

Done when: You have evidence of actual demand, recurring topics and a plausible delivery process.

Phase 2 — Requirements and service policies

P0

* Write the product requirements document.

* Define learner, administrator and future support roles.

* Specify every request field.

* Define ticket statuses and permitted transitions.

* Specify existing-answer matching and duplicate handling.

* Define notification events and preferences.

* Write content quality standards.

* Define privacy, copyright, moderation and acceptable-use policies.

* Document private-service scope, revisions, cancellations and refunds.

* Specify analytics events and success metrics.

Why: This converts the idea into instructions a designer and developer can implement without guessing.

Tools: Requirements document, workflow diagrams, policy drafts and an acceptance-criteria checklist.

Done when: Each important feature has a defined behavior, permission rule and testable completion condition.

Phase 3 — UX, UI and accessibility

P0

* Map every learner journey.

* Map the creator/admin workflow.

* Design the navigation and information architecture.

* Wireframe the landing page and request form.

* Design the request detail page and status timeline.

* Design the tutorial library and search results.

* Design the notification centre.

* Design private quote and checkout screens for the later paid phase.

* Define reusable colors, typography, spacing and components.

* Design loading, empty, error and offline states.

* Test the prototype with five target users.

* Check keyboard navigation, contrast, labels and mobile usability.

Why: Users should understand what to do next without needing an explanation from you.

Tools: Figma or an equivalent design tool, accessibility checks and prototype testing.

Done when: Representative users can complete the key journeys in the prototype, and serious usability problems have been addressed.

Phase 4 — Technical architecture and integrations

P0

* Choose the frontend and backend architecture.

* Select the database and migration system.

* Choose authentication.

* Choose a transactional email provider.

* Define public and private video hosting.

* Verify payment provider eligibility for your business and intended markets.

* Choose analytics, error monitoring and file storage.

* Diagram data flows and external dependencies.

* Set up development, staging and production environments.

* Define secret management, backups and recovery.

Why: These choices affect cost, security, delivery reliability and how much future development will be

required.

Tools: A web framework, PostgreSQL or an equivalent database, authentication, email delivery, video hosting, payment APIs and monitoring.

Done when: The architecture is documented, integrations are selected based on verified capabilities, and the development environment can be reproduced.

Phase 5 — Database and backend foundations

P0

* Create user and account-state models.

* Create request and visibility models.

* Create tutorial reply and video metadata models.

* Record status history.

* Create notification and delivery-attempt models.

* Create topic and tag models.

* Implement server-side authorization.

* Validate all incoming data on the server.

* Add rate limits and abuse controls.

* Add safe error handling and audit logs.

* Create database migrations and test data.

Why: This is the foundation that keeps tickets connected to the right users, replies, notifications and permissions.

Tools: Database, backend APIs, authentication middleware, validation library, logging and rate-limiting systems.

Done when: The core data model works and automated tests prove that users cannot access or change other people's private requests.

Phase 6 — Build the core MVP

P0 · Primary build phase

* Build the landing page.

* Implement signup, login, verification and password reset.

* Build the request submission form.

* Show confirmation after successful submission.

* Build the learner's My Requests dashboard.

* Build the request detail page.

* Build the admin request queue.

* Implement request status transitions.

* Build the admin reply composer.

* Validate tutorial links and access labels.

* Publish a reply against the original request.

* Build the public tutorial library.

* Add basic search, topic filters and pagination.

* Suggest existing tutorials before users submit duplicate requests.

Why: These features create the smallest complete version of your product.

Tools: Frontend, backend, database, authentication and YouTube or another chosen video host.

Done when: A learner can submit a request, an administrator can process it and attach a tutorial, and the learner can open that reply from their own request page.

This is the first major release milestone. Everything else should support or improve this journey.

Phase 7 — Notifications and communication

P0

* Define all notification-triggering events.

* Create email templates with plain-text alternatives.

* Send submission confirmations.

* Notify learners when replies are ready.

* Build an in-app notification centre.

* Implement background processing and retries.

* Handle invalid addresses and delivery failures.

* Add appropriate email preferences and unsubscribe handling.

* Test notification links on mobile and desktop.

* Prevent duplicate or misleading messages after retries.

Why: A completed tutorial is of little use if the requester never learns that it is ready.

Tools: Transactional email service, notification database tables, background job queue and monitoring.

Done when: Successful ticket events reliably create notifications, and failed deliveries can be detected and retried.

Phase 8 — Private requests and payments

P1 · After the public MVP is reliable

* Define a private request brief.

* Create a quoting and scope-acceptance process.

* Verify payment provider eligibility, fees and settlement arrangements.

* Build secure checkout.

* Verify payment using signed server-side webhooks.

* Record transactions and payment states.

* Handle cancelled and failed payments.

* Implement refunds and dispute workflows.

* Protect private deliverables with an appropriate access-control system.

* Test duplicate payment events, delayed events and refunds.

* Introduce paid priority only if capacity and queue policies justify it.

Why: Paid requests create revenue, but also create financial, privacy and customer-service responsibilities.

Tools: Payment provider, webhook handler, transaction database, restricted video/file delivery and admin tools.

Done when: A customer can agree to a defined service, pay successfully, receive the promised deliverable and obtain support for legitimate payment or delivery problems.

Critical: Never treat a browser redirect to a success page as proof that a payment has cleared. Verify payment server-side.

Phase 9 — Content operations and teaching quality

P0

* Document the repeatable tutorial production process.

* Create a teaching brief for each accepted request.

* Define a quality-review checklist.

* Check technical accuracy and software-version relevance.

* Remove personal information from examples used publicly.

* Standardize video titles, descriptions and resource links.

* Add accurate captions and transcripts where feasible.

* Collect learner feedback.

* Establish a correction process for outdated or incorrect videos.

* Prioritize new topics using demand and production effort.

Why: Your competitive advantage is not simply publishing videos. It is creating videos that solve the learner's actual problem.

Tools: YouTube Studio or your selected video host, a content workflow, quality checklists and learner feedback.

Done when: Every published tutorial is linked to the correct request, reviewed against defined standards and accompanied by enough context for the learner to use it.

Phase 10 — Security, privacy and trust

P0 · Required before launch

* Threat-model accounts, tickets, attachments and payments.

* Apply least-privilege administrator permissions.

* Protect API keys and webhook secrets.

* Secure attachments and validate external URLs.

* Define data minimization and retention rules.

* Implement appropriate account export and deletion workflows.

* Create content reporting and moderation tools.

* Review applicable privacy, consumer protection and tax obligations.

* Establish an incident response process.

* Run dependency and vulnerability checks.

Why: Learning requests may contain personal information, business documents or other sensitive context. Trust must be designed into the product rather than added after a problem occurs.

Tools: Access controls, secret management, secure file storage, logging, privacy documentation and security testing.

Done when: Access boundaries are tested, data-handling rules are documented and critical security issues are resolved.

Phase 11 — Quality assurance

P0

* Write unit tests for validation and status changes.

* Test the full ticket-to-reply lifecycle.

* Test user permissions and private-ticket isolation.

* Test notification retries and failures.

* Test payments in a sandbox before enabling real transactions.

* Test mobile layouts and major browsers.

* Test keyboard navigation and accessibility.

* Test slow connections and interrupted sessions.

* Test database restoration.

* Conduct a formal launch-readiness review.

Why: A feature that works in one demonstration is not necessarily reliable in real use.

Tools: Automated test framework, browser testing, payment sandbox, accessibility tooling and staging environment.

Done when: Critical user journeys pass repeatable tests, high-risk defects are resolved and the release has explicit approval.

Phase 12 — Deployment and operations

P0

* Create the source-code repository.

* Establish branch protection and code review.

* Configure automated tests and build checks.

* Separate staging and production environments.

* Configure the domain, HTTPS and security headers.

* Configure email-domain authentication.

* Set up uptime and error alerts.

* Create a support inbox and response templates.

* Write operational runbooks.

* Deploy a private beta.

* Run live smoke tests.

* Define rollback and release procedures.

Why: Launching the app is not the same as operating it. You need to know when something breaks and how to recover.

Tools: Source control, hosting platform, CI/CD, domain/DNS, monitoring, email and database backup services.

Done when: A real production user can complete the primary workflow and you can detect, diagnose and recover from common failures.

Phase 13 — Launch, marketing and growth

P1

* Create a launch landing page with real examples.

* Recruit a small pilot cohort.

* Seed the library with useful starter tutorials.

* Create a search and content-distribution strategy.

* Establish sharing and referral rules.

* Review the first 30 days of product data.

* Test improvements to form clarity and conversion.

* Review request backlog and creator capacity weekly.

* Add learning paths when enough related content exists.

* Evaluate additional creators or subject experts.

* Test additional revenue models only when justified by evidence.

Why: Growth must not overwhelm your ability to deliver quality tutorials.

Tools: Website analytics, YouTube, relevant communities, search optimization, user feedback and capacity reports.

Done when: You have a repeatable way to attract relevant learners and can serve them without an uncontrolled backlog.

Phase 14 — Analytics and continuous improvement

Ongoing

* Instrument the main user journey with privacy safeguards.

* Build an operational dashboard.

* Calculate production cost and margin for paid tutorials.

* Measure tutorial helpfulness and learning outcomes.

* Review the product and risk register monthly.

* Improve search based on failed searches and duplicate requests.

* Establish infrastructure usage and cost alerts.

* Revisit the roadmap using real evidence.

Why: The app should become better at solving learning problems over time, not simply accumulate more features.

Tools: Product analytics, operational dashboards, cost tracking, learner surveys and issue tracking.

Done when: Product decisions are informed by observed user outcomes, service quality and sustainable operating costs.

4\. The end-to-end acceptance checklist

This is the single most important test for the MVP. Before launching publicly, verify each step with real accounts and real notifications.

MVP release gate

0/10

Learner can register, verify their email and log in.

Authentication works, including recovery from a forgotten password.

Learner can submit a useful tutorial request.

Validation works and the ticket is stored with the correct owner and visibility.

The platform can suggest existing answers.

Public tutorials appear when relevant; private content never leaks into public results.

Administrator can review and process the ticket.

Authorized status changes are recorded in the history.

Administrator can attach a tutorial reply.

The link is valid, the tutorial matches the request, and access claims are accurate.

Learner is notified of the reply.

The notification arrives and its link opens the correct request.

Learner can access the tutorial and provide feedback.

The tutorial is usable and feedback is recorded against the correct request.

Security boundaries are verified.

Another learner cannot read or modify the ticket, reply or private files.

Failures can be detected and recovered from.

Email failures, application errors and backup recovery have been tested.

Policies and support are ready.

Users can understand the terms, report issues and reach the responsible operator.

Reset checksCopy checklist

These are interactive planning checks in this conversation, not a live connection to your codebase or test environment.

5\. The recommended technology approach

I would favor a relatively simple architecture that minimizes maintenance while still supporting private requests and payments later.

Layer	Suggested starting point	Purpose
Frontend and server	Next.js with TypeScript	Web interface and server-side application logic
Database	PostgreSQL	Users, tickets, replies, statuses and transactions
Authentication	Managed authentication or a well-maintained framework solution	Accounts, sessions and verification
Email	Transactional email provider	Confirmations and reply notifications
Public videos	YouTube	Public tutorial publishing and discovery
Private delivery	Hosting with suitable access controls	Restricting access to paid deliverables
Payments	A provider verified for your business location	Checkout, payment confirmation and refunds
Hosting	A managed deployment platform	Production hosting and releases
Monitoring	Error tracking and uptime monitoring	Detecting failures
Analytics	Privacy-conscious product analytics	Understanding usage and learning outcomes

These are recommendations, not integrations that have already been created. We should verify current provider eligibility, pricing, regional support and access-control capabilities before locking in the stack.

6\. Important things not to build too early

To protect your time and budget, I would initially defer:

* A native Android or iOS app.

* A complex AI recommendation engine.

* A marketplace with multiple independent creators.

* Live chat and real-time collaboration.

* Membership subscriptions and elaborate pricing tiers.

* Automated video creation and editing.

* Advanced gamification and leaderboards.

* A complicated points or rewards economy.

Start with the smallest version that reliably turns a real request into a useful learning outcome. Add sophistication when real usage demonstrates that it is needed.

7\. Your immediate next actions

These are the next five tasks I recommend doing before committing to the full technical build.

1. Confirm the initial audience

Choose whom to serve first and which digital skills to prioritize.

2. Collect real requests

Use a simple form and gather actual learning needs.

3. Run a manual pilot

Deliver a few tutorial requests using your existing tools.

4. Finalize the product requirements

Use the results to define screens, workflows, policies and acceptance criteria.

5. Select the stack and begin the MVP

Choose services only after the requirements and constraints are clear.

Copy next actions

My strongest advice is to treat the spreadsheet as the master backlog, the acceptance checklist as the release gate, and the real learner feedback as the authority for what gets built next. The goal is not to complete the most tasks. It is to build a trustworthy service that helps people learn, delivers on its promises, and can grow sustainably.

Honestly, I think this is a really strong idea, and the clarification you just made makes it significantly better. 🤔

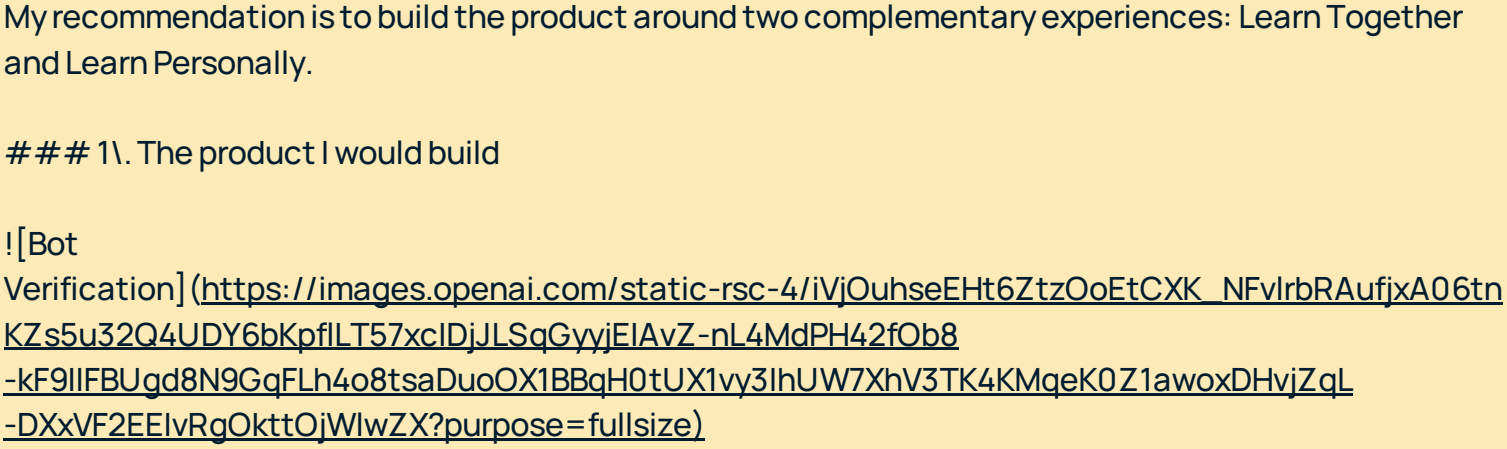
You're not just building an app where people request tutorials. You're building a learning platform where people tell you what they want to learn, and the platform helps turn those requests into actual learning resources.

That creates a potentially powerful cycle: one person's question becomes a tutorial that can help

hundreds or thousands of other people, while people with unique needs can pay to receive a tutorial made specifically for them.

My recommendation is to build the product around two complementary experiences: Learn Together and Learn Personally.

1\. The product I would build



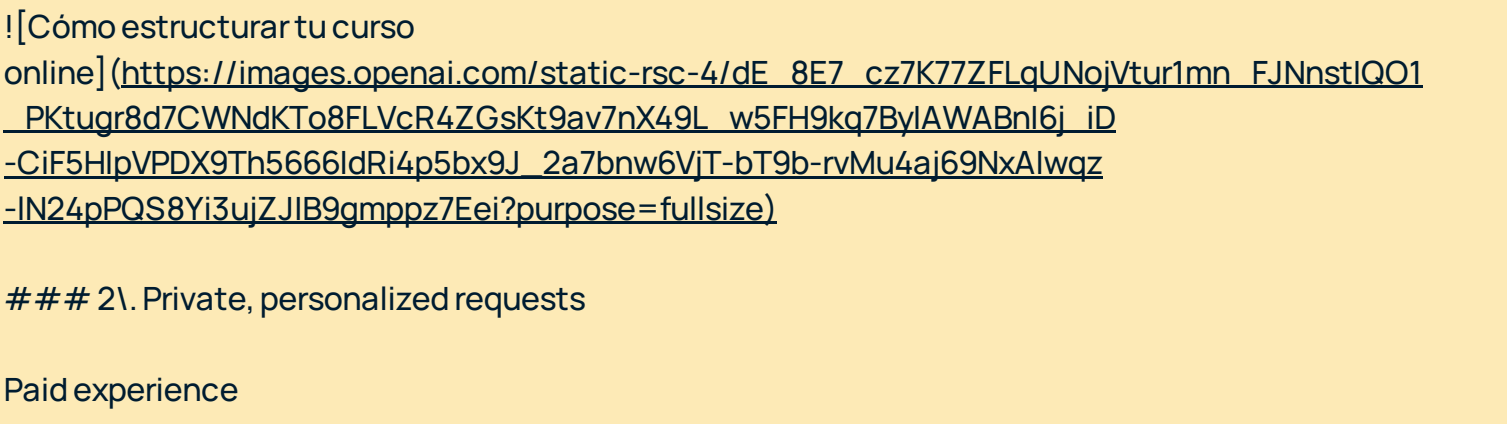
1\. Public learning requests

Core experience

Someone requests a tutorial, such as "How do I design a professional flyer using Canva?"

You create the video, publish it publicly on YouTube, and attach the link to the request. The requester gets notified, and the tutorial becomes available to everyone.

The benefit: One video can solve the same problem for many learners, making your work increasingly valuable over time.



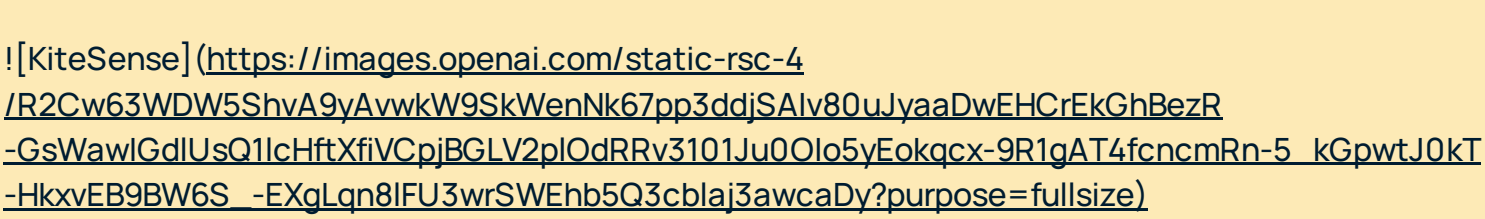
2\. Private, personalized requests

Paid experience

Someone requests a tutorial made specifically for them, perhaps using their own business documents, a particular project, or a skill they need to learn privately.

They see the price, confirm the scope, pay, and receive the completed tutorial through their private request page.

The benefit: You can earn directly from specialized teaching without needing to make every tutorial public.



3\. A growing searchable learning library

Long-term growth

Every suitable public tutorial becomes part of a searchable library organized by skill, topic, difficulty, and software.

Learners can discover existing answers before creating another request.

The benefit: The platform becomes more useful with every completed public tutorial.

2\. The most important product decision: don't make people pay twice for the same answer

I would design the rules like this:

Situation	What the learner gets	Payment model
The answer already exists publicly	Immediate access to the tutorial	Free
Someone requests a new public tutorial	A place in the public request queue; the resulting video is accessible to everyone	Free by default
Someone wants a new tutorial made privately	A custom tutorial with agreed deliverables	Paid
Someone wants a public request prioritized	Optional paid priority, if you decide to offer it	Paid, clearly disclosed
Someone wants personal help beyond a video	Additional guidance or consultation	Optional paid service

One important refinement: don't charge people just to submit ordinary learning requests at the beginning. Your early advantage will come from learning what people need and attracting enough requests to identify recurring problems.

You can introduce paid options after you have evidence that people want them.

And if someone pays for a private tutorial, don't simply promise that a YouTube link will be private. YouTube's unlisted videos can be shared by anyone who obtains the link. For genuinely restricted access, you will need an appropriate hosting and access-control arrangement, or you must clearly explain the limitations of the chosen hosting option.

3\. How the app should work from beginning to end

Learner submits a request

Skill, goal, current level, desired outcome and public/private preference

System checks for existing answers

Recommend matching tutorials or let the learner submit a genuinely new request

Public request

Queue, community interest, production and public YouTube upload

Private request

Scope, quote, payment confirmation and restricted delivery

Tutorial is delivered

Video link, helpful notes, completion status and relevant learning resources

Learner gets notified

In-app notification and email, with a direct link to the request and tutorial

Feedback improves the platform

Was the tutorial helpful? What remains unclear? What should be taught next?

4\. Features I would add to make the idea much stronger

These are the improvements I would prioritize beyond the initial concept.

A demand counter for each request

When multiple learners want the same tutorial, let them support an existing request instead of creating duplicates. Display the number of interested learners and notify them together when the tutorial is published.

An existing-tutorial matcher

Before someone submits a request, search the platform's library. If an answer already exists, show it immediately. If it only partially answers the question, allow the person to request the missing part.

A genuinely useful request ticket

Ask what they want to accomplish, which tool they use, their skill level, and what a successful result looks like. Allow screenshots or example files when appropriate, with privacy warnings. This helps you make the right video the first time.

A transparent request tracker

Use clear statuses: Submitted → Under review → Planned → In production → Published → Completed. For private paid work, include Awaiting payment, Payment confirmed and Awaiting learner approval where appropriate.

Learning outcomes, not just video links

Each tutorial should have a clear learning objective, a short description, any necessary resources, and optionally a practice task. Ask whether the learner actually achieved their goal.

Trust, privacy and quality controls

Establish rules for harmful requests, copyright, personal information, inappropriate uploads, refund disputes, misleading content and private-video access. Verify payment server-side before treating a paid request as funded.

5\. How I would make money without undermining the mission

I would start with a simple model, then expand when demand justifies it.

Free community learning

Public tutorials and public requests drive discovery, trust, and audience growth.

Paid custom tutorials

Charge according to complexity, research required, video length, turnaround time and any included follow-up support. Agree on scope before payment.

Premium learning packages

Later, sell structured learning paths, downloadable project files, exercises, or bundles of tutorials.

Business and organizational training

Eventually, businesses could pay for tailored tutorials to teach their employees specific software, workflows or internal processes.

If you operate from Nigeria, payment integration should be chosen around your actual business location, settlement currency, fees, payout arrangements and supported payment methods. Don't assume that every international payment provider or feature will be available to you.

6\. What I would build first, and what I would deliberately postpone

The biggest risk is spending months developing an elaborate platform before confirming that people will submit requests and watch the resulting videos.

Here is my recommended implementation order.

* Phase 1 — Validate the demand

  Interview potential learners, identify the first target audience, collect real tutorial requests, and discover which topics recur most often.

  Done when: You have real requests and a shortlist of high-demand topics.

* Phase 2 — Design the product and workflow

  Define public versus private requests, ticket statuses, permissions, notification rules, video delivery, payment policies and acceptance criteria.

  Done when: Every important user journey has a clear, testable definition.

* Phase 3 — Build the MVP

  Implement accounts, request tickets, an existing-tutorial library, an admin dashboard, video-link replies, and email/in-app notifications.

  Done when: A learner can submit a request and successfully receive and open its tutorial reply.

* Phase 4 — Pilot public tutorials

  Process real requests, publish useful videos, gather feedback and improve the matching system.

  Done when: Learners use the product repeatedly and completed videos demonstrably address their requests.

* Phase 5 — Introduce private paid requests

  Add quotes, explicit scope acceptance, payment processing, verified payment webhooks, secure delivery, refunds and dispute handling.

  Done when: A private request can be purchased and fulfilled safely from beginning to end.

* Phase 6 — Optimize and scale

  Add demand aggregation, analytics, search improvements, creator capacity planning, SEO, referrals and eventually additional instructors.

  Done when: The platform can handle more requests without quality, trust or response times deteriorating.

7\. The metrics that will tell you whether the app is succeeding

Don't measure success only by the number of registered users or videos uploaded.

Metric	What it tells you
Request-to-tutorial completion rate	Whether requests actually get fulfilled
Median time to first meaningful response	Whether learners feel acknowledged
Median time to tutorial delivery	Whether your production capacity meets demand
Existing-answer match rate	How effectively the library prevents duplicate work
Tutorial helpfulness score	Whether videos solve the original problem
Repeat learner rate	Whether people find lasting value in the platform
Cost and revenue per private request	Whether paid work is sustainable
Notification delivery and click-through rate	Whether learners actually receive and use their replies

Set initial targets after observing a small pilot. You can then establish realistic service commitments based on how many videos you can produce per week.

My final recommendation

Build the public learning-request system first, but design the architecture to support private paid requests from day one.

The public side creates reach and a growing library. The private side creates a direct revenue opportunity. Both serve the same fundamental purpose: helping people learn what they actually need, rather than forcing them to search endlessly for a tutorial that may not exist.

One final product principle matters enormously: the platform must deliver value even when you haven't made a new video yet. Existing tutorials, request updates, community demand signals and clear expectations will make the experience useful while learners wait.

And I would make the entire product revolve around one promise:

Tell us what you want to learn. We'll help you find it, or work toward creating it.

That is a much stronger proposition than simply being a place to submit tutorial requests.

I wanna create a webapp where people can make a request of the kind of thing or digital skill they'll love to learn..they'll create it like a ticket and then I'll create a video for them to learn it may on YouTube and then I'll submit the link into the app as a reply to their request and they'll be notified of the reply to their request.

as an expert in all conceivable roles that are needed to come together to make this app work completely to help it fulfill its purpose and make it work exceedingly well, i need you to scrutinize, optimize, improve, the app to ensure its gonna get to its highest possible potential and meet its audience at its point of needs as intended

ok now create a todo list that covers the implementation roadmap, its should be ultra-granular, covering every itsy bitsy step needed, state whats been done, whats next, why its been done, what its to acheive, what its gonna use/connect to in the process of each step, etc and every other conceivable thing i didnt mention that the todo list should have for its caliberas an expert in all conceivable roles that are needed to come together to make this app work completely to help it fulfill its purpose and make it work exceedingly well, i need you to scrutinize, optimize, improve, the app to ensure its gonna get to its highest possible potential and meet its audience at its point of needs as intendedlets create an implementation roadmap for the app from start to finish as an expert in all conceivable roles that are needed to come together to make this app work completely to help it fulfill its purpose and make it work exceedingly well, i need you to scrutinize, optimize, improve, the app to ensure its gonna get to its highest possible potential and meet its audience at its point of needs as intendedWell if the video is already available then its gonna be accessible to other users unless it's a private tutorial requested for only them and that's gonna have like payments and confirmation for stuff like that.. What do you think 🤔